MacEwan UNIVERSITY

CMPT 315

Lab 5 – DOM Manipulation & Events

General Information

- **Submission due date**: Due **@10:59 AM on 20-Oct-2026** (right before your next lab)
- **Lab weight**: 2%

OVERVIEW:

In this lab, we will build a web page with an interactive User Interface (UI) and a JavaScript validated form that allows members to join the **Tavel Photo Club**. This page utilizes the following concepts that will be covered in lecture:

- The Document Object Model (DOM)
- Modifying the DOM
- Events and Event Handling
- Event Types
- Forms in JavaScript

PART 1: Setting Up

Complete these steps before you start coding.

1. Create a folder: travel-club
2. In VS Code: File -> Open Folder -> select travel-club
3. Create files: **index.html**, **style.css** and **script.js**

PART 2: index.html

*Copy/paste the following into **index.html**, then complete TODO item (HTML #1).*

HTML

```
<!-- ======================================================
    Lab 5 - index.html starter
====================================================== -->

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Travel Photo Club Registration</title>
    <link rel="stylesheet" href="style.css">
```

```
</head>

<body>
    <header>
        <h1>Join the Travel Photo Club</h1>
    </header>

    <main>

        <section id="gallery">
            <h2>Featured Shots (Select to Learn More)</h2>
            <div class="gallery-grid">
                <div class="thumb" tabindex="0" data-city="Paris">
                    <img alt="Paris travel photo"
                    src="https://images.unsplash.com/photo-1502602898657-3e91760cbb34?w=800"/>
                </div>

                <!-- TODO (HTML #1): Add the remaining thumbnails (Tokyo, New York, etc. at
                    least 3) using real images from Unsplash / Pexels / Pixabay. Each thumbnail
                    must contain an <img> and must keep tabindex="0" on the .thumb container.
                -->

            </div>
            <p id="image-caption"> Click (or press Enter) to select.</p>
        </section>

        <section id="registration">
            <h2>Member Sign Up</h2>
            <form id="signup-form" novalidate>
                <div class="form-group">
                    <label for="username" id="label-1">Username:</label>
                    <input type="text" id="username" name="username">
                    <span class="error-msg" id="error-username"></span>
                </div>
                <div class="form-group">
                    <label for="email">Email:</label>
                    <input type="email" id="email" name="email">
                    <span class="error-msg" id="error-email"></span>
                </div>
                <div class="form-group">
                    <label for="password">Password:</label>
                    <input type="password" id="password" name="password">
                    <span class="error-msg" id="error-password"></span>
                </div>
                <div class="form-group">
                    <label>
                        <input type="checkbox" id="terms"> I agree to the terms
                    </label>
                    <span class="error-msg" id="error-terms"></span>
                </div>
                <button type="submit">Register Now</button>
            </form>
        </section>

    </main>

    <script src="script.js"></script>

</body>

</html>
```

Your gallery section can use online images from one of the following sources:

- Unsplash
- Pexels
- Pixabay

PART 3: style.css

*Copy/paste the following into **style.css**.*

CSS

```
/* ======================================================
    Lab 5 - style.css starter
====================================================== */

body {
    font-family: sans-serif;
    background-color: #f4f4f4;
    padding: 20px;
}

section {
    background: white;
    padding: 20px;
    margin-bottom: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

/* Form Styles */
.form-group {
    margin-bottom: 15px;
}

label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

input[type="text"],
input[type="email"],
input[type="password"] {
    width: calc(100% - 20px);
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
    transition: border-color 0.3s ease;
    transition: background-color 0.3s ease;
}

.error-msg {
    color: red;
    font-size: 0.9em;
    display: none;
}

.error-border {
```

```
    border-color: red !important;
    background-color: #fff0f0;
}

/* Gallery Styles */
.gallery-grid {
    display: flex;
    gap: 10px;
}

.thumb {
    width: 100px;
    height: 100px;
    background-color: #ddd;
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    position: relative;
    overflow: hidden;
    transition: transform 0.3s ease-out;
    transition: background-color 0.3s ease-out;
}

.thumb img {
    width: 100%;
    height: 100%;
    object-fit: cover;
    display: block;
}

/* Dynamic Class */
.expanded {
    background-color: #60b946;
    color: white;
    transform: scale(1.1);
}
```

PART 4: script.js

*After creating **script.js**, implement the following required features.*

1. JavaScript Feature — DOM Ready

Goal: Your script must not run too early. Ensure the DOM is ready before selecting elements and attaching event listeners. If done correctly, you should not see console errors such as “Cannot read properties of null”.

API hints:

- `document.addEventListener("DOMContentLoaded", ...)`

2. JavaScript — Focus/Blur UX on Inputs

Goal: When a user enters a field, the field becomes visibly active. When they leave, it returns to normal.

Inputs to target:
- Username
- Email
- Password

Required behavior:
- On focus: background becomes `#fffae0`
- On blur: background returns to white

Constraints:
- Select the inputs using one `querySelectorAll(...)` call with a combined selector.
- Attach listeners to each input.

API hints:
- Event types: `focus`, `blur`
- Attach events: `addEventListener`
- Style update allowed here: `element.style.backgroundColor = ...`

3. JavaScript Feature — Form Validation on Submit

Goal: Stop form submission when inputs are invalid. Show clear error messages beside each field and highlight invalid fields.

Validation rules (must implement all):

Field	Rule	Error message (shown beside field)
Username	Required (not empty after trimming spaces)	Username is required
Email	Required + must look like an email	Enter a valid email address
Password	Length ≥ 8	Password must be at least 8 characters
Terms Checkbox	Must be checked	You must accept the terms

Error display requirements:
When a field is invalid:

- Put the message into the correct span using `textContent`.
- Make the `<span>` visible (it is hidden by default).
- Add the class `error-border` to the associated input (or checkbox area).

When a field becomes valid:

- Clear the `<span>` text.
- Hide the `<span>` again.
- Remove `error-border`.

Submission behavior:

- Your submit handler must call `preventDefault()` to prevent page reload.
- If everything is valid, show a success message (either an `alert()` or a message inserted into the page).
- Do not send data anywhere (client-side validation only).

Constraints (important):

- Do not use `innerHTML` for error messages. Use `textContent`.
- Do not use `alert()` for errors. Errors must appear beside fields.
- Validation must run on submit.

Email validation note:
Use a simple approach: a basic regex OR a minimal check such as requiring both "`@`" and "`.`" in sensible positions.

API hints:

- Form event: `submit`
- Stop reload: `event.preventDefault()`
- Read fields: `input.value` and `.trim()`
- Checkbox state: `checkbox.checked`
- Add/remove classes: `element.classList.add(...)`, `element.classList.remove(...)`
- Show/hide spans: update `element.style.display` or toggle a class you define

4. JavaScript Feature — Gallery Interaction

Click behavior:

- Select tiles using `querySelectorAll(".thumb")`.
- Clicking a tile expands only that tile (single selection) and removes expansion from the others.
- Update the caption to: "`This shot is from CityName`", where `CityName` comes from `data-city` (recommended) or from the image `alt` text.
- Update caption using `textContent` and apply/remove .expanded using `classList`.

API hints:

- Click event: `click`
- Identify clicked tile: event target or this (in a regular function handler)
- Update caption: `captionEl.textContent = ...`
- Single selection logic: remove .expanded from all tiles, then add to the clicked tile

Keyboard support:

Thumbnails have `tabindex="0"` which makes them focusable. `tabindex=0` means the element becomes keyboard-focusable, even if it isn't normally (like a `<div>`). You will add support so that pressing Enter on a focused tile selects it (same behavior as click).

API hints:

- Event: `keydown`
- Check key: `event.key === "Enter"`

PART 5: Local Testing + Deployment

Before you deploy your website on GitHub, you must first confirm it works locally. Once local testing succeeds you will go ahead and deploy. You can follow these steps:

1. **Save your files**
    - Windows: **Ctrl + S**
    - Mac: **Cmd + S**
2. **Test Option A (Highly Recommended): Use Live Server in VS Code**
    - Right-click index.html → **Open with Live Server**
    - Your browser will open with a local address like http://127.0.0.1:5500/
    - Save changes and the page updates automatically (or refresh if needed)
3. **Test Option B (Simple): Open the file directly**
    - In VS Code, right-click index.html
    - Choose **Reveal in File Explorer** (Windows) / **Reveal in Finder** (Mac)
    - Double-click index.html to open it in your browser
4. **Focus/Blur Checklist**
    - Focus makes background yellow for Username, Email, and Password.
    - Blur returns background to white
5. **Form Validation Checklist**
    - Empty username shows correct message and red border.
    - Invalid email shows correct message and red border.
    - Password under 8 characters shows correct message and red border.
    - Terms unchecked shows correct message.
    - Valid fields clear errors and remove red border.
    - Form does not reload when invalid.
    - Success behavior happens only when all fields are valid.
6. **Interactive Gallery Checklist**
    - Clicking a tile expands only one tile at a time.
    - Caption updates with correct city name.

- Scale and background effects are smooth (transition works).

7. **Deploy to the Internet**
   - Refer to previous labs or use your preferred steps to **go live** on GitHub Pages.

SUBMISSION

Submit **one PDF document** through mêskanâs containing your answers to the questions below:

1. Paste your **GitHub Repo URL** (e.g., https://github.com/username/travel-club)
2. Paste your live **GitHub Pages URL** (e.g., https://username.github.io/travel-club)
3. What are **selection methods** and what is their purpose? Did you use any in this lab?
4. Is it **innerHTML** or **textContent** that is the preferred property for modify the content of a given element? What is the reasoning behind this?
5. What is an **event handler** in JavaScript and what method is used to register event handlers?

DEMO: Tuesday, October 20, 2026 @11:00 AM

- Demonstrate your completed lab during the lab session.
- Be prepared to explain your solution and answer questions about your code.

MARKING SCHEME

Component	Mark
Lab Questions (PDF Submission)	5
Lab Demo	15
Total	**20**